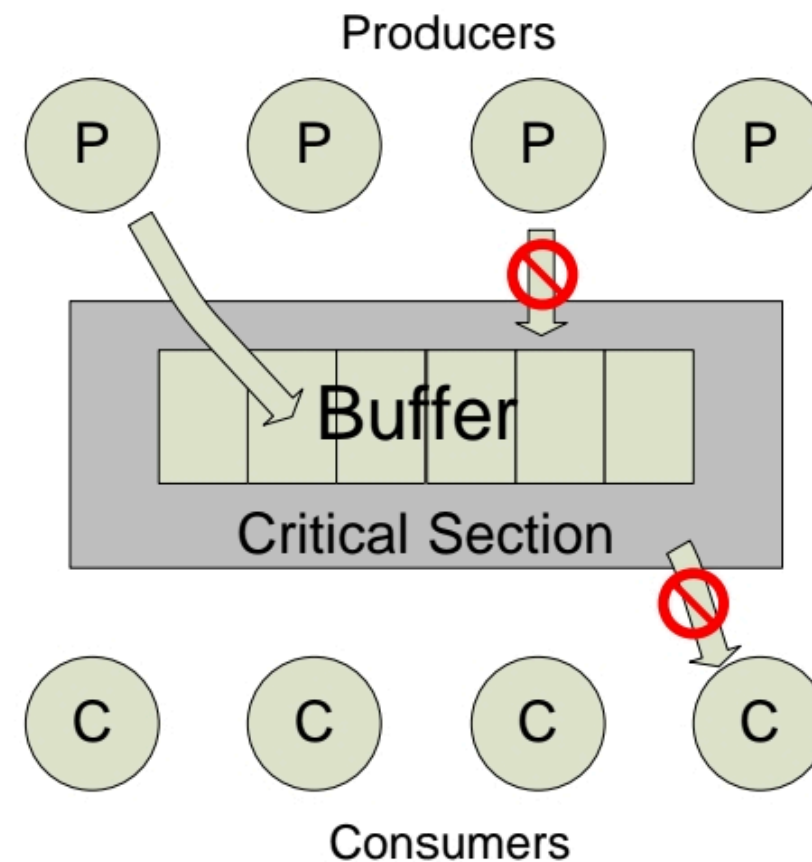


Synchronization

공유자원의 위험성



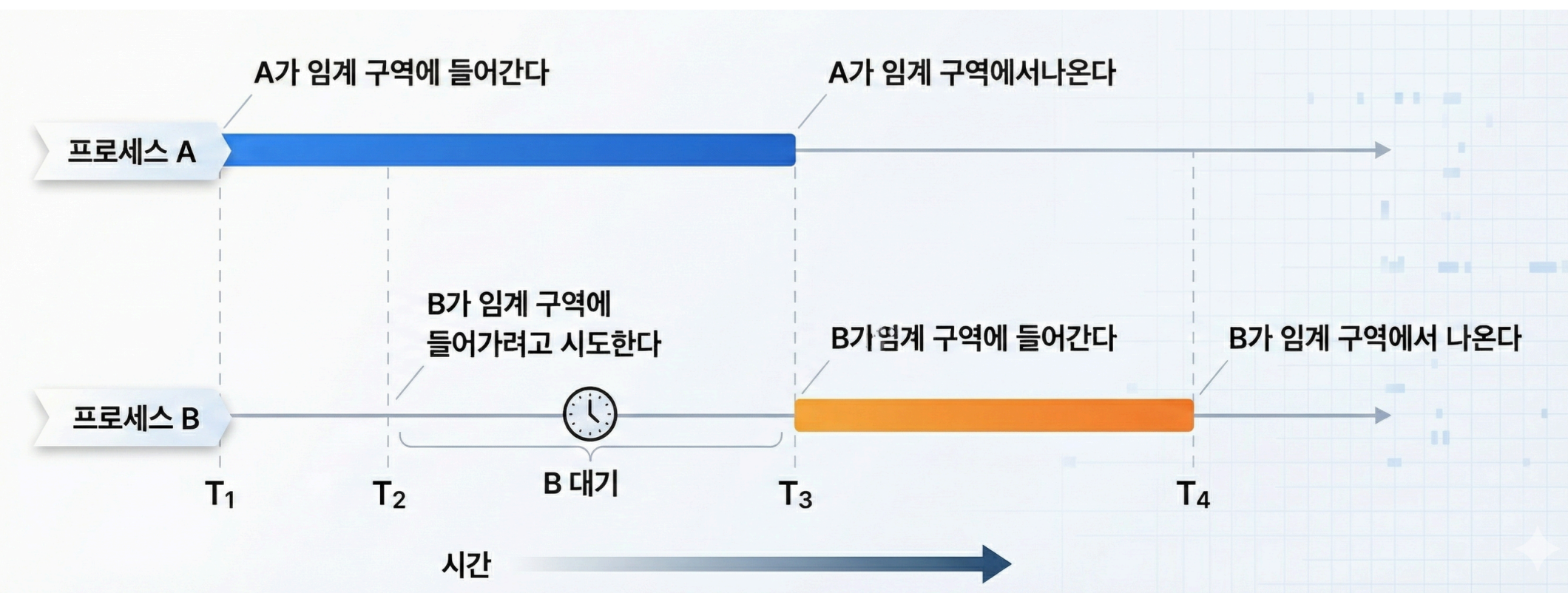
bounded buffer problem

- 변수 X에 더하기와 뺄셈을 하는 상황
- $x += 300$ 은 사실
 - x를 load
 - 레지스터에서 더한후..
 - 다시 x에 store
- 연산 중간에 누가 수정하면?
- 동기화 필요:
 - 실행 순서(Execution Order):올바른 순서대로 접근
 - 상호 배제(Mutual Exclusion):자원에 하나의 프로세스만 접근

Race Condition(경쟁 상태)

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

- Race Condition: 프로세스(스레드)가 공유 자원에 동시에 접근하고, 그 접근 순서에 따라 실행 결과가 달라지는 상황
- Critical Section: 동시에 실행하면 문제되는 공유 데이터에 접근하는 코드 영역



임계구역 해결의 3가지 조건

Process P_0

```
flag [0] = true;
turn = 1;
while (flag [1] && (turn == 1)) ;
    critical section
flag [0] = false;
    remainder section
```

Process P_1

```
flag [1] = true;
turn = 0;
while (flag [0] && (turn == 0)) ;
    critical section
flag [1] = false;
    remainder section
```

Peterson's Solution

- SW 기반의 동기화의 문제점:
- 복잡하고, 느려짐
 - N개의 프로세스라면 더욱 심화
 - 지루하고 현학적

- **Mutual Exclusion**: 한 프로세스가 임계 구역에 진입했다면 다른 프로세스는 임계 구역에 들어 올 수 없음.
- **Progress**: 임계 구역에 어떤 프로세스도 진입하지 않았다면 들어갈 수 있어야함.
- **Bounded Waiting** : 프로세스는 언젠가는 임계 구역에 들어올 수 있어야 한다 (=무한정 대기x).

Synchronization Hardware

Process P_0

```
flag [0] = true;
turn = 1;
while (flag [1] && (turn == 1)) ;
    critical section
flag [0] = false;
    remainder section
```

```
do {
    acquire lock
        critical section
    release lock
        remainder section
} while (TRUE);
```

- 하드웨어의 지원을 받아 Atomic(방해, 분리 불가)한 연산 가능
 1. 락을 건다.
 2. 락을 푼다.

SW로 해결이안되면 HW의 도움을받자

- 빠름
- N개의 프로세스라도 가능
- 단순하고 기억에남음

그러나 Mutual exclusion만 쉽게 해결.
나머지 2가지는 SW로 해결해야함.

Mutex lock(spinlock)

```
do {  
    acquire lock  
        critical section  
    release lock  
        remainder section  
} while (TRUE);
```

```
• acquire() {  
    while (!available)  
        ; /* busy wait */  
        /* 계속 기다리기 때문 */  
    available = false;;  
}  
  
• release() {  
    available = true;  
}
```

뮤텍스 특징:

- 락이 한개
- 락을 건 놈이 풀어야함. (락의 소유권 존재)
- Busy wait: while 문안에서 계속 체크 (빙글빙글 도는것처럼 보여서 spinlock)

(counting) Semaphore

P1	P2	P3
wait() // 임계 구역 signal()	wait() // 임계 구역 signal()	wait() // 임계 구역 signal()

자원이 여러개(하나이상*)여도 가능한 동기화 방법

- 정수 변수 S = 남은 자원의 수 = 동시에 몇 개의 스레드까지 작업?
- wait() 또는 P(): 기다리기 (S 가 남아있다면 바로작업)
- signal() 또는 V(): 신호주기

실행 순서 동기화

task1	wait
signal	task2

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

* $S=0,1$ 이면 binary semaphore

Mutex VS Semaphore

무텍스

소유권 있음

자원 1개

세마포어

소유권 없음

자원 N개

busy wait?

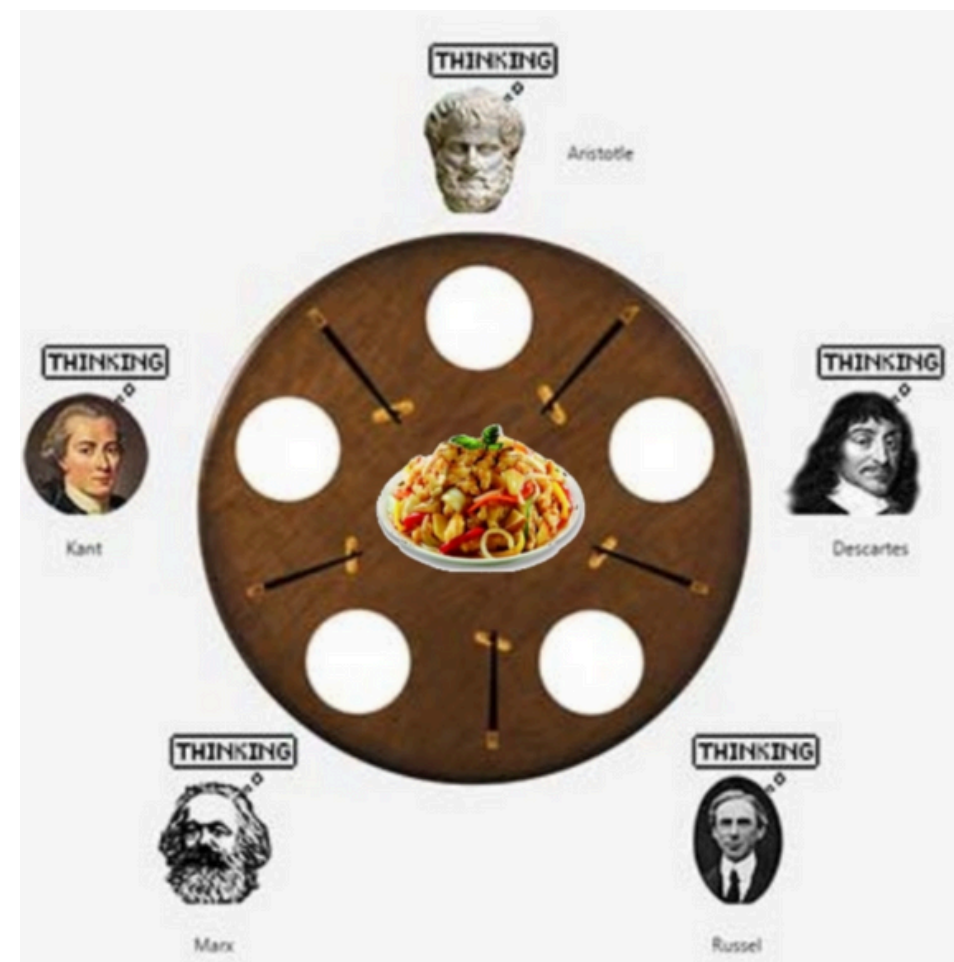
Busy wait이 유리한 경우? (당연한 전제:멀티코어)

- critical section의 길이가 짧은경우 → Busy wait이 유리
- 길고 짧은 건 뭘 기준?
 - context switching보다 짧다
 - IO, Network : CPU기준 억겁의 시간



DEADLOCK(교착상태)

두 개 이상의 프로세스가 서로 상대방이 가진 자원을 기다리며 무한 대기하는 상태



- 식사하는 철학자 문제
- 서로의 포크(젓가락?)이 필요

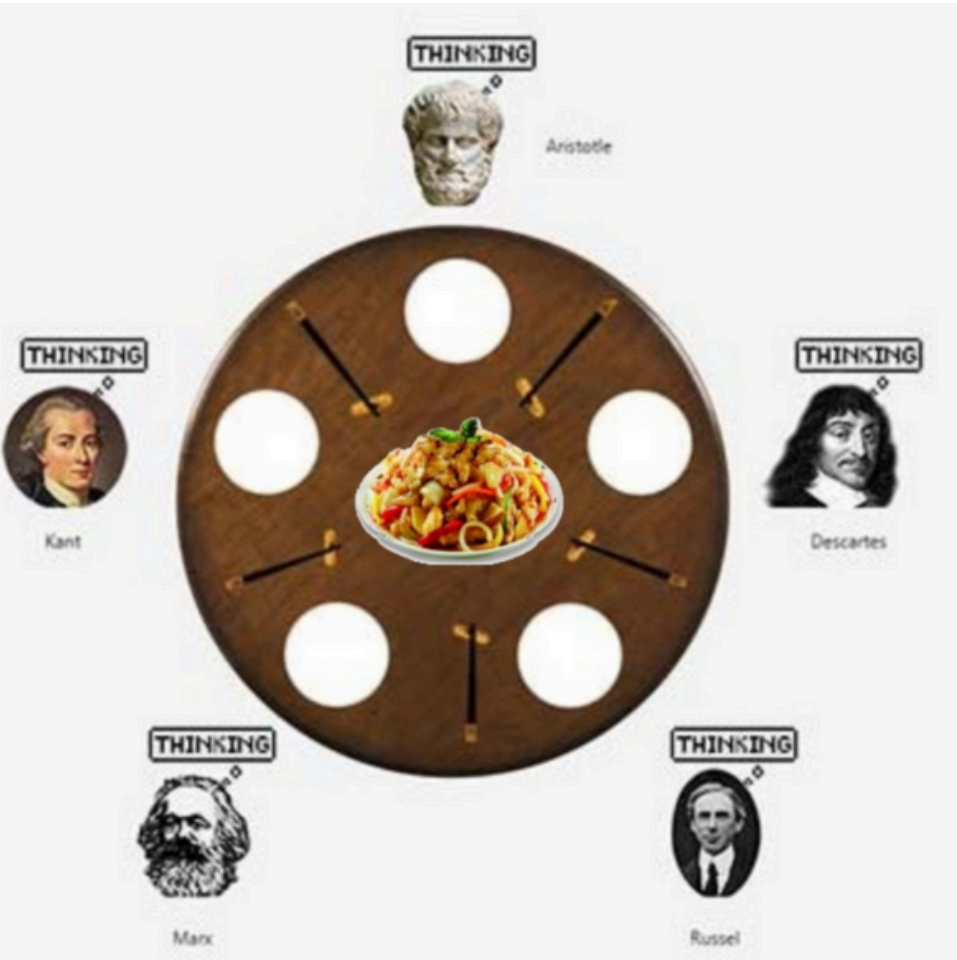
데드락의 필요조건:

- 상호 배제
- 점유 상태로 대기
- 선점 불가
- 순환성 대기

- 포장을 뜯는데 가위가 필요함 - 가위를 얻으려면 포장을 ...
- 지원자는 회사의 일을 원함 - 회사는 지원자의 경력을 ...

DEADLOCK 조건

데드락의 4가지 조건



상호 배제(Mutual exclusion):	자원은 하나의 프로세스만 사용 가능함 (자원 공유 불가)
점유 상태로 대기 (Hold and Wait)	이미 자원을 가진 프로세스가 다른 자원을 기다릴 때(wait), 보유 자원을 놓지 않고 계속 가지고 있음.(hold)
선점 불가(no preemption)	프로세스는 자원을 강제로 빼앗기지 않음
순환성 대기(Circular wait)	자원을 기다리는 프로세스 간에 cycle 존재

4조건이 모두 성립해야 발생 가능

DEADLOCK 해결방법

1. 예방: 하나라도 충족하지 않으면 발생하지 않으니 하나를 없애자

상호 배제(Mutual exclusion):	자원을 공유한다(비현실적)
점유 상태로 대기 (Hold and Wait)	자원을 모두 얻은 상태에서 작업 자원을 요청할때는 자원이 전혀 없을때만 요청가능
선점 불가(no preemption)	강제로 뺏는다 상태 저장과 복구가 쉬운 CPU,Memory같은 자원이라면 가능
순환성 대기(Circular wait)	자원에 순서를 부여하고 오름차순으로 요청

2. 회피:

- 실행환경에서 추가적인 정보를 활용해 데드락 발생 회피
 - 시스템 상태가 안전한 상태로 돌아올 수 있는 경우에만 자원 할당
- >모든 프로세스가 실행 전에 평생 사용할 자원의 최대량을 미리 알아야.. (안씀)

DEADLOCK 해결방법

3. 탐지 및 회복: 데드락 발생은 허용하되, 그에 대한 탐지 루틴을 두어 데드락 발견시 회복

- 프로세스 종료
- 일시적인 선점 허용

4. **무시**: 개발자가 알아서 하겠지~

- 1,2,3 방법들이 현실성이없거나, 비효율적이라 자원낭비가 심함.
- 교착상태가 자주 발생하는 현상이 아님, 이 현상을 주기적으로 검사해야하나?

Q&A

질의 응답

